

Code Explanation

Testing the Transform Module

The provided code is a test suite for the transform module, which is responsible for cleaning, transforming, and enriching sales data. The tests ensure that the functions in the transform module work as expected.

Overview of the Code

The code consists of three test functions: ``test_clean_and_transform_data``, ``test_enrich_sales_data``, and ``test_add_sales_metrics``. Each test function tests a specific function from the transform module. The tests use Pytest and PySpark to verify the correctness of the data transformation and enrichment process.

Step 1: Testing the `clean_and_transform_data` Function

The ``test_clean_and_transform_data`` function tests the ``clean_and_transform_data`` function from the transform module. This step checks if the function correctly cleans and transforms the sales data.

```
def test_clean_and_transform_data(sample_sales_data):  
    result_df = clean_and_transform_data(sample_sales_data)  
    assert isinstance(result_df, DataFrame)  
    assert "total_amount" in result_df.columns  
    assert "year" in result_df.columns  
    assert "month" in result_df.columns  
    assert "day" in result_df.columns
```

Step 2: Verifying Null Handling in `clean_and_transform_data`

This step checks if the ``clean_and_transform_data`` function correctly handles null values in the sales data.

```
assert result_df.filter("quantity IS NULL").count() == 0  
assert result_df.filter("unit price IS NULL").count() == 0
```

Step 3: Checking Calculations in `clean_and_transform_data`

This step verifies if the ``clean_and_transform_data`` function correctly calculates the total amount for each sale.

```
row = result_df.filter("sale_id = 'S001']").first()  
assert row["total amount"] == row["quantity"] * row["unit price"]
```

Step 4: Testing the `enrich_sales_data` Function

The ``test_enrich_sales_data`` function tests the ``enrich_sales_data`` function from the transform module. This step checks if the function correctly enriches the sales data with product and customer information.

```
def test_enrich_sales_data(sample_sales_data, sample_products_data,
sample_customers_data):
    transformed_df = clean_and_transform_data(sample_sales_data)
    result_df = enrich_sales_data(transformed_df, sample_products_data,
sample_customers_data)
    assert isinstance(result_df, DataFrame)
    assert "product_name" in result_df.columns
    assert "category" in result_df.columns
    assert "customer_name" in result_df.columns
    assert "region" in result_df.columns
```

Step 5: Verifying Join Correctness in enrich_sales_data

This step checks if the `enrich\_sales\_data` function correctly joins the sales data with product and customer data.

```
row = result_df.filter("sale_id = 'S001']").first()
assert row["product_name"] == "Product 1"
assert row["customer_name"] == "Customer 1"
assert row["region"] == "East"
```

Step 6: Testing the add_sales_metrics Function

The `test\_add\_sales\_metrics` function tests the `add\_sales\_metrics` function from the transform module. This step checks if the function correctly adds sales metrics to the enriched sales data.

```
def test_add_sales_metrics(sample_sales_data, sample_products_data,
sample_customers_data):
    transformed_df = clean_and_transform_data(sample_sales_data)
    enriched_df = enrich_sales_data(transformed_df, sample_products_data,
sample_customers_data)
    result_df = add_sales_metrics(enriched_df)
    assert isinstance(result_df, DataFrame)
    assert "product_total_sales" in result_df.columns
    assert "customer_total_sales" in result_df.columns
    assert "daily_region_rank" in result_df.columns
```

Step 7: Verifying Window Function Calculations in add_sales_metrics

This step checks if the `add\_sales\_metrics` function correctly calculates the sales metrics using window functions.

```
product1_rows = result_df.filter("product_id = 'P001']").collect()
assert len(product1_rows) > 0
product1_total = product1_rows[0]["product_total_sales"]
for row in product1_rows:
    assert row["product_total_sales"] == product1_total
```